One-Hour-Ahead PM2.5 Forecasting Using Machine Learning

Animesh Kumar Rai Rishiwant Kumar Maurya Supreet Aditya Shankar Drishti Jha
Team T051 — Advance Machine Learning (Phase 1)
Roll numbers: 2401010069, 2401020102, 2401010465, 2401010038, 2401010162

Abstract

Forecasting fine particulate matter (PM2.5) helps people and authorities reduce exposure to polluted air. In this project we predict the PM2.5 concentration in Beijing one hour ahead using the public UCI Beijing dataset (2010–2014, 43,824 hourly records). We treat it as a supervised learning problem: from the recent pollution and weather history we predict the next hour's value. A central challenge is that PM2.5 changes slowly, so the simple rule "the next hour equals the current hour" — the persistence baseline — is already very accurate and hard to beat. To keep the comparison fair, all models are scored on exactly the same hours and the evaluation is strictly chronological, so no information from the future reaches the training stage. We build 37 features from past data and train an XGBoost model, tuning it only on 2013 and testing it once on 2014. XGBoost lowers the test error from 11.93 to 10.96 $\mu g\,m^{-3}$ (MAE) and from 22.03 to 19.96 $\mu g\,m^{-3}$ (RMSE), roughly an 8–9% improvement over persistence; it is better in all twelve test months and the improvement is statistically significant. The gain comes mainly at low-to-moderate pollution, while during extreme haze the model does not beat persistence. Deep learning is reserved for Phase 2.

Index Terms

PM2.5, air quality forecasting, machine learning, XGBoost, gradient boosting, persistence baseline, time-series validation.

I. Introduction and Problem Statement

Fine particulate matter (PM2.5) is one of the most harmful air pollutants for human health. The 2021 World Health Organization guideline recommends a 24-hour mean below 15 $\mu g\,m^{-3}$ [1], yet Beijing frequently exceeds this by a wide margin. Short-term forecasts of PM2.5 are useful because they give people and local authorities time to react — for example, to issue warnings or limit outdoor activity.

In this project we study a focused version of the problem: given the pollution and weather observed up to the current hour, can we predict the PM2.5 concentration one hour later more accurately than simple baseline rules? This one-hour horizon is deliberately short and realistic for the data we have. It is also surprisingly hard to improve on, because PM2.5 is highly autocorrelated: the value in the next hour is usually close to the value now. The naive rule of carrying the current value forward (persistence) is therefore a strong competitor, and any machine learning model is only worth reporting if it clearly beats such baselines under an evaluation that does not accidentally use future information.

Objectives. The project has four practical goals:

1) prepare and validate the dataset, and build useful input features from past observations only;
2) train and tune a machine learning model (XGBoost) and compare it fairly against persistence and a seasonal-naive baseline;
3) evaluate the models chronologically, with the test year used only once, and check whether the improvement is statistically meaningful;
4) analyse where the model helps and where it struggles, and state the limitations honestly.

Scope. This report covers the Phase 1 machine learning work only. Deep learning models, such as recurrent neural networks, are part of Phase 2 and are not implemented or evaluated here.

II. Literature Review

Bai et al. [2] review air pollution forecasting and group the methods into statistical, machine learning, and numerical (chemical transport) approaches, along with hybrids. Our work belongs to the machine learning family and uses only station-level history and observed weather.

The dataset was introduced by Liang et al. [3], [4], who studied how weather affects PM2.5 in Beijing. A useful lesson from their analysis is that raw year-to-year pollution comparisons can be misleading because weather conditions differ between years, so evaluation must be done carefully.

For modelling, Zhong et al. [5] used a gradient-boosting method (LightGBM) to estimate hourly PM2.5 across China from weather data, and Li et al. [6] applied an LSTM network to multi-station Beijing data. These studies motivate both tree ensembles and neural networks for this task. Gradient boosting itself was formalised by Friedman [7], and XGBoost [8] is an efficient, regularised implementation that also handles missing values directly — a property we rely on.

On evaluation, Tashman [9] and Bergmeir and Benítez [10] warn against ordinary (random) cross-validation for time series and recommend chronological, out-of-sample testing; Hyndman and Athanasopoulos [11] stress the importance of naive benchmarks. We follow this guidance. To test whether two forecasts differ in accuracy we use the Diebold–Mariano test [12] with a Newey–West variance estimate [13]. Compared with these works, our contribution is a careful, leakage-controlled comparison of XGBoost against strong naive baselines on a single station, with an honest analysis of when the model fails.

III. Dataset and Exploratory Data Analysis

The dataset [4] contains 43,824 hourly records from 1 January 2010 to 31 December 2014. The target variable is PM2.5 ($\mu g\,m^{-3}$) measured at the US Embassy in Beijing. The input variables, recorded at Beijing Capital International Airport, are dew point, temperature, pressure, combined wind direction (NE, NW, SE, and calm/variable), cumulative wind speed, and the cumulative hours of snow and of rain. To guarantee that we always analyse the same data, the raw file is kept read-only and its SHA-256 checksum is verified every time it is loaded.

Data quality. The hourly timeline is complete, unique, and in order. Only the PM2.5 column has missing values: 2,067 hours (4.7%) spread over 214 gaps that range from a single hour to 155 hours. Missing data is more common in 2010–2012 (6–8%) than in 2013–2014 (about 1%). The highest recorded values (up to 994 $\mu g\,m^{-3}$) match known pollution episodes, such as around Chinese New Year, so they are genuine measurements and are kept rather than removed.

Key observations from EDA. PM2.5 is strongly right-skewed: most hours are low, but there is a long tail of severe-pollution hours (median 72, 99th percentile 420 $\mu g\,m^{-3}$), as shown in Fig. 1. A logarithmic transform makes the distribution much more symmetric. The series is also highly autocorrelated: the correlation between consecutive hours is 0.96, dropping to 0.39 at a 24-hour lag and to about 0.02 at one week. The partial autocorrelation is negligible beyond a lag of two hours (Fig. 2), which tells us that only the most recent hours carry strong predictive information. Finally, wind matters: north-westerly wind is associated with clean air (median 31 $\mu g\,m^{-3}$), while calm or variable wind is associated with high concentrations (median 98 $\mu g\,m^{-3}$). These observations directly guided our feature and model choices.

IV. Data Preprocessing and Feature Engineering

Preprocessing. Our preprocessing is deliberately conservative. We build a complete hourly time index, check the data invariants (for example, that pressure, temperature, and dew point lie in physically sensible ranges), and keep every genuine value. We do not fill in the missing PM2.5 values, because filling a gap would require using later observations and would leak future information; instead, we simply do not score any model on hours where the values it needs are missing. We also do not delete the extreme-pollution hours, since those are exactly the cases that matter most. The cumulative wind-speed column adds up speed while the wind

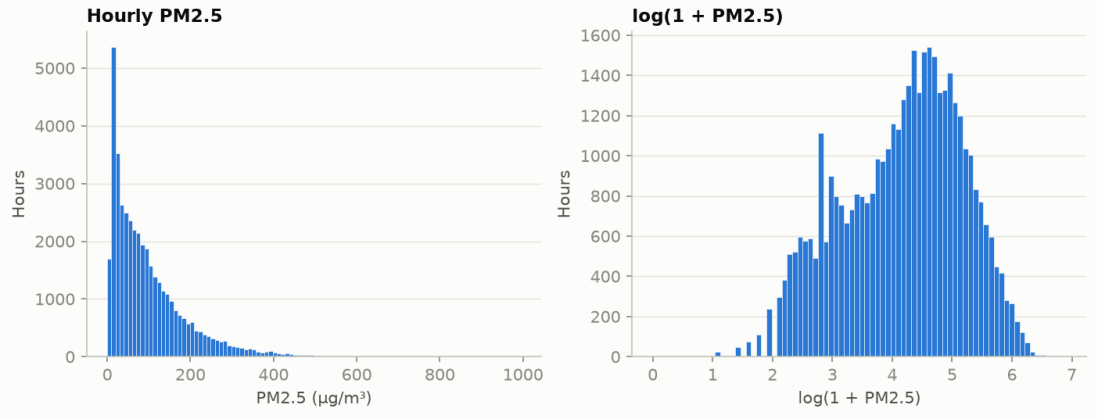


Fig. 1. Distribution of hourly PM2.5 (left) and of $\log(1 + \text{PM2.5})$ (right). The raw values are strongly right-skewed with a long tail; the log transform is much more symmetric.

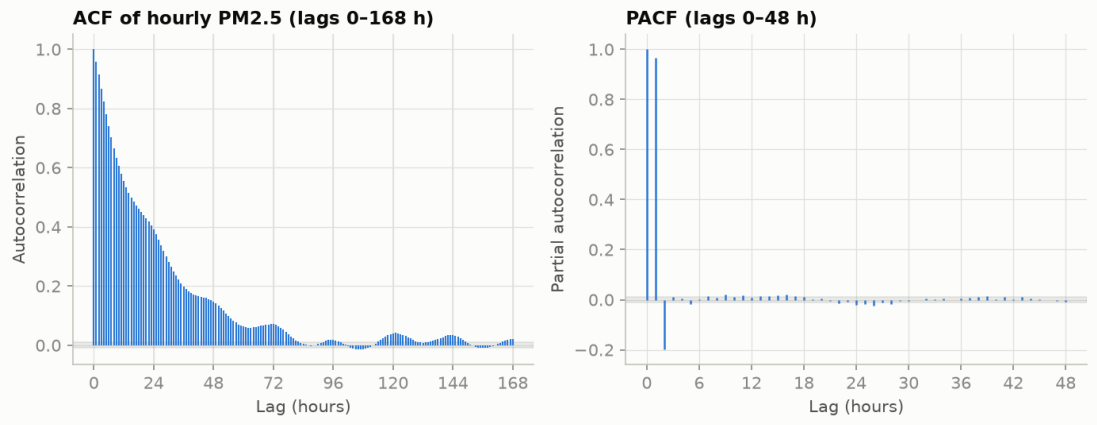


Fig. 2. Autocorrelation (left) and partial autocorrelation (right) of hourly PM2.5. The strong correlation at short lags is why recent values are the best predictors.

direction stays the same and resets when the direction changes; we decode it back into an hourly wind speed by taking the increment within each run of constant direction.

Feature engineering. From the past observations we build 37 features, grouped in Table I. They fall into three families: (i) PM2.5 history — recent lagged values, rolling averages and standard deviations over 3–24 hours, the last hourly change, and how many hours have passed since the last valid reading; (ii) weather — dew point, temperature, pressure, the temperature–dew-point spread, 3-hour tendencies, the decoded wind speed, how long the current wind direction has persisted, a one-hot encoding of wind direction, and the snow/rain counters; and (iii) calendar — the hour and month (encoded as sine/cosine pairs) and the day of the week. The lag choices follow directly from the autocorrelation analysis.

Preventing data leakage. Every feature uses only information available at the forecast time: values at the current hour or earlier, plus calendar facts about the next hour (which are always known in advance, such as whether it will be 3 a.m. or a Sunday). To confirm there is no accidental leakage, we ran a temporal leakage check: at 30 random points in time we replaced all data after that point with random numbers and verified that every feature value at or before that point stayed exactly the same. Since corrupting the future never changed any feature, each feature depends only on past and current data, as intended.

TABLE I
The 37 engineered features, all computed from data at or before the forecast time.

Group	Examples	Count
PM2.5 history	lags, rolling mean/std, last change, staleness	17
Weather	dew point, temperature, pressure, decoded wind	15
Calendar	cyclical hour and month, day of week	5

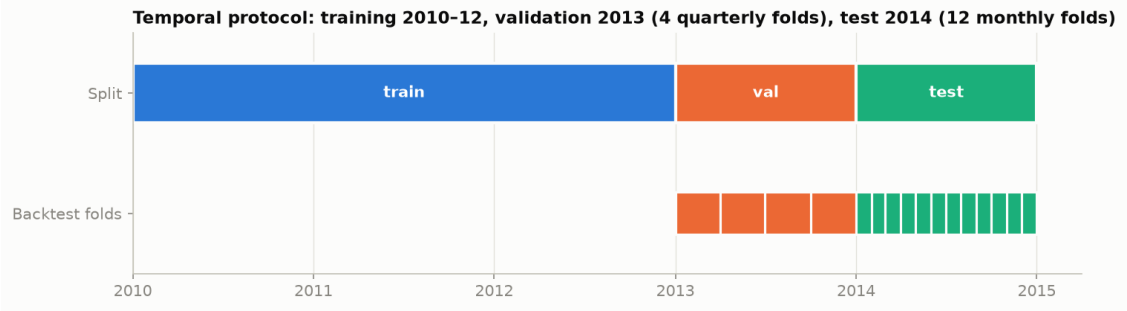


Fig. 3. Chronological protocol. The model is tuned on the four quarters of 2013 and finally evaluated month by month on 2014, always training only on earlier data.

V. Methodology and Machine Learning Models

A. Problem formulation

We issue the forecast at the end of hour t and predict the value one hour later:

$$y_t = x_{t+1}, \tag{1}$$

where x_{t+1} is the PM2.5 concentration at hour $t+1$. A model may use inputs from hours $\leq t$ and calendar attributes of $t+1$; it may not use the actual future weather. Each sample is assigned to a split according to its target hour, so that a training example never predicts into the validation or test period. We split by whole calendar years — training on 2010–2012, validating on 2013, and testing on 2014 (Fig. 3) — which keeps every season in each split.

All models are scored on a common evaluation set: the hours where the target and both baseline inputs are available (23,452 training, 8,563 validation, and 8,531 test hours). Scoring every model on identical hours makes the comparison fair.

B. Baseline models

Two naive baselines set the bar. Persistence predicts $\hat{y}_t = x_t$: the next hour simply equals the current hour. Because consecutive hours are correlated at 0.96, this is a very strong forecast. Seasonal-naive predicts $\hat{y}_t = x_{t+1-24}$: the same hour on the previous day. This baseline turns out to be poor — in fact worse than predicting the overall average — because the day-to-day pattern repeats only weakly (the 24-hour correlation is just 0.38). Reporting both baselines shows that a learned model must do more than exploit a simple daily cycle.

C. The XGBoost model

XGBoost [8] builds an ensemble of regression trees, where each new tree is trained to correct the errors left by the trees before it, with regularisation that discourages overly complex trees. It is well suited to this kind of tabular data: it captures non-linear interactions between weather and pollution history, handles missing inputs directly, and needs no feature scaling.

TABLE II
Selected XGBoost configuration (chosen on the 2013 validation folds).

Hyperparameter	Value
Prediction target	change ($x_{t+1} - x_t$)
Maximum tree depth	6
Learning rate	0.02
Number of trees	800
Minimum child weight	10
Row subsample	0.85
Column subsample	0.6
L_2 regularisation (λ)	5

One practical choice mattered: what the model should predict. Instead of predicting the level x_{t+1} directly, we found it worked better to predict the change $x_{t+1} - x_t$ and then add the current value back. The intuition is that regression trees output a constant on each leaf, so asking them to reproduce "roughly the current value" wastes many splits; learning the usually small change is easier. We tried three options — the level, $\log(1 + x_{t+1})$, and the change — and the change formulation gave the lowest validation error (Section VI). We do not claim this is necessary for every tree-based model; it was the best of the three formulations in our experiments.

D. Tuning

We selected hyperparameters by the lowest average error over four expanding-window folds (the quarters of 2013), trying 40 random configurations. The chosen settings are listed in Table II. The 2014 test year was not touched until the configuration was frozen. In the final evaluation we retrain the model before each month of 2014 using all earlier data (12 folds); a single fit on 2010–2013 is also reported for comparison.

E. Evaluation metrics

We report three standard regression metrics. Writing $e_t = y_t - \hat{y}_t$ for the forecast error over the n scored hours,

$$\mathrm{MAE} = \frac{1}{n}\sum_t |e_t|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_t e_t^2}, \tag{2}$$

$$R^2 = 1 - \frac{\sum_t e_t^2}{\sum_t (y_t - \bar{y})^2}. \tag{3}$$

MAE is the average size of the error and is easy to interpret. RMSE squares the errors, so it is more sensitive to the large mistakes made during heavy-pollution hours. R^2 measures how much better a model is than always predicting the mean $\bar{y}$; a value near 1 is good, and a negative value means the model is worse than the mean. Finally, to check that the improvement is not just luck, we use the Diebold–Mariano test, which formally asks whether two forecasts have genuinely different accuracy; because hourly errors are correlated in time, we use an autocorrelation-robust variance for the test.

VI. Experimental Results and Discussion

A. Choosing the model on validation (2013)

On the 2013 validation folds, predicting the level or the log did not beat persistence (mean RMSE 23.79 and 24.31 versus 23.70 $\mu g\,m^{-3}$), whereas predicting the change did (21.80 $\mu g\,m^{-3}$); tuning lowered this to 21.41. Repeating the fit with five random seeds changed the RMSE by only about 0.04 $\mu g\,m^{-3}$, so the improvement is stable rather than a lucky run. Pooled over 2013, XGBoost reached MAE 12.26 and RMSE 22.53 $\mu g\,m^{-3}$, compared with 13.18 and 24.87 for persistence.

TABLE III
Test-year results (2014, 8,531 common hours). MAE and RMSE in $\mu g\,m^{-3}$.

Model	MAE	RMSE	R^2
Persistence	11.93	22.03	0.944
Seasonal-naive (24 h)	67.30	98.75	−0.121
XGBoost (change)	10.96	19.96	0.954

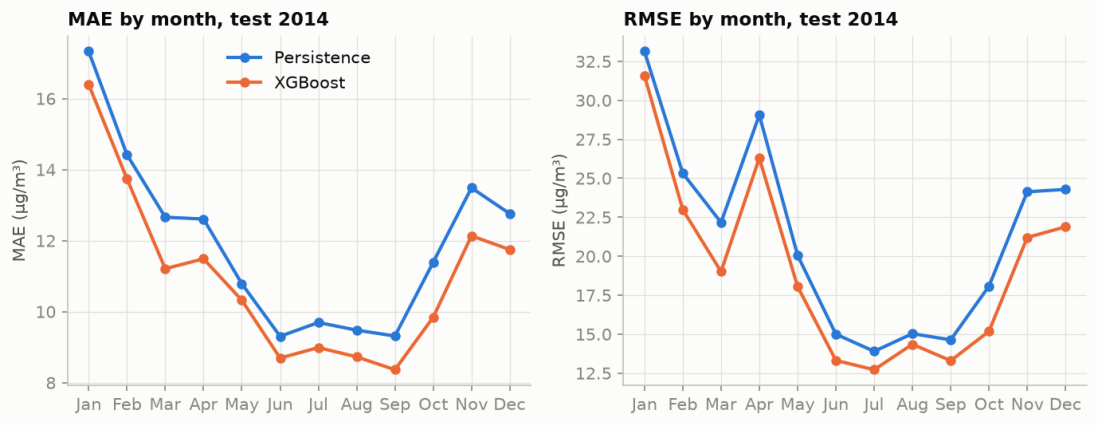


Fig. 4. Monthly MAE and RMSE on the 2014 test year. XGBoost (orange) stays below persistence (blue) in every month.

B. Test-year results (2014)

Table III shows the single, final evaluation on 2014. XGBoost improves on persistence by 8.1% in MAE and 9.4% in RMSE, and it has the lower error in all twelve months on both metrics (Fig. 4). The seasonal-naive baseline has a negative R^2, confirming that a plain daily cycle is not a useful forecast. The Diebold–Mariano test strongly rejects the idea that XGBoost and persistence are equally accurate ($p \approx 3.4 \times 10^{-7}$ for squared-error loss), so the improvement is very unlikely to be due to chance. Retraining every month gave essentially no benefit over a single fit (RMSE 19.96 versus 19.97), which means a simpler one-time training would work almost as well here.

C. Error analysis

Performance across pollution levels. Grouping the test hours by their true PM2.5 level (Fig. 7) shows where the model helps. XGBoost reduces RMSE in every band up to 250 $\mu g\,m^{-3}$, but on the most extreme hours — above 250 $\mu g\,m^{-3}$, which are only 7.6% of the test set — it is slightly worse than persistence (42.0 versus 41.6) and tends to under-predict the peak by about 12.8 $\mu g\,m^{-3}$ on average. These few hours account for 34% of its total squared error. This happens because the trees average the target on each leaf, which pulls rare, very high values down toward more typical ones.

Actual versus predicted. Fig. 5 shows a test episode. Both models track the general shape of the pollution curve, but XGBoost reacts slightly earlier to falling concentrations, while it lags and under-shoots the sharp rise into 16 January. More generally, when pollution drops quickly (often as north-westerly wind brings in clean air, which the model can see in its inputs), XGBoost improves clearly; when pollution builds up suddenly, there is little warning in the inputs and no model does well.

What drives the predictions. We measured feature importance two ways: by permutation (shuffling a feature and seeing how much the error grows) and by the model's internal gain. Both agree (Fig. 6) that the current PM2.5 value and the decoded hourly wind speed are the most useful inputs, followed by the 3-hour change in dew point. This matches the EDA and basic physics: recent pollution is the strongest signal, and wind clears the air. We note that these are predictive associations, not proof of cause and effect. As a further check, the

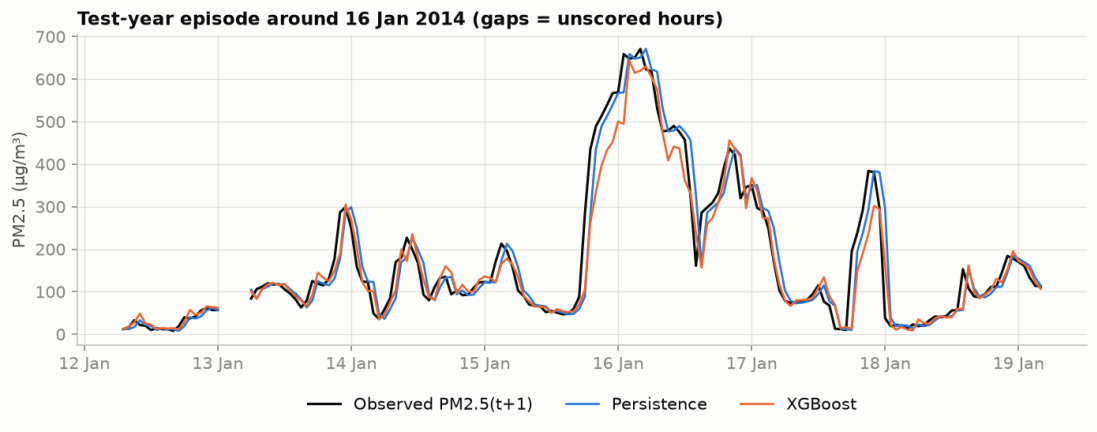


Fig. 5. Observed versus forecast PM2.5 during a 2014 test episode. XGBoost tracks the fall in concentration but under-predicts the peak around 16 January.

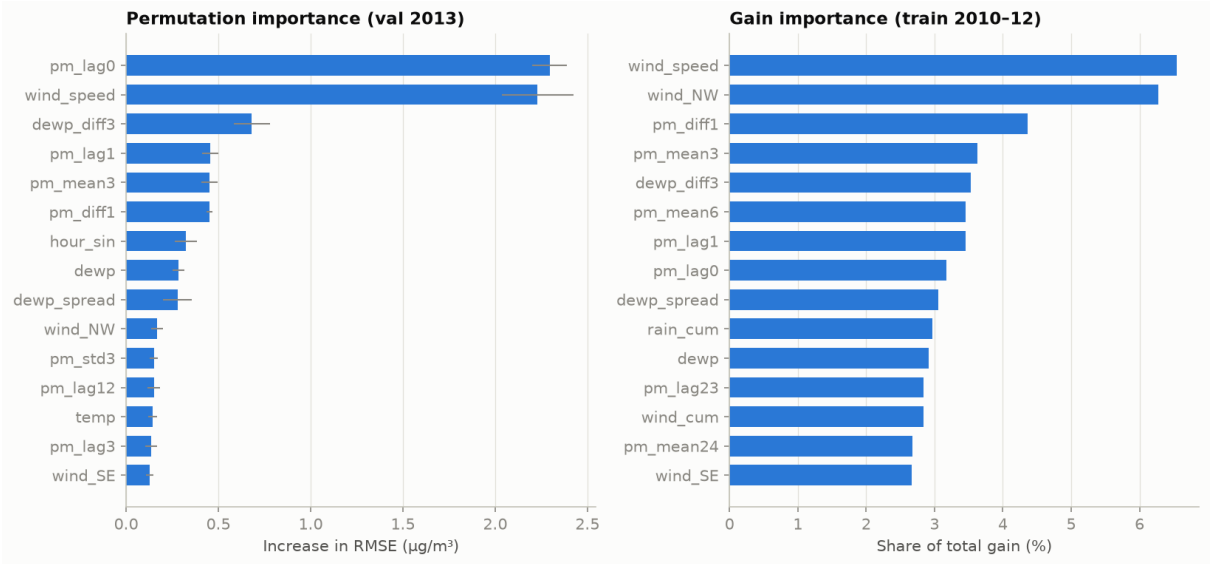


Fig. 6. Feature importance for XGBoost, by permutation on the 2013 validation data (left) and by internal gain on the training data (right). Recent PM2.5 and the decoded wind speed dominate.

model's residuals are much less autocorrelated than those of persistence (lag-1 autocorrelation 0.003 versus 0.143), meaning XGBoost has extracted most of the easy short-term structure.

D. Discussion

The overall picture is that machine learning gives a real but modest improvement here, which is expected: when the next hour is already 96% correlated with the current one, there is limited room to improve on simply carrying the value forward. Two findings stand out. First, how we framed the prediction (predicting the change rather than the level) mattered more than fine-tuning the model. Second, the model's skill is one-sided: it anticipates clean-air arrivals because the responsible wind is visible in the inputs, but it cannot foresee sudden pollution build-ups, which have no clear early signal in the available data. These are useful, honest lessons about both the method and the problem.

VII. Limitations and Future Scope

Limitations. The results are based on a single monitoring station and a single test year, and 2014 appears to be an easier year than 2013 for both models, so the exact numbers should not be over-generalised. The

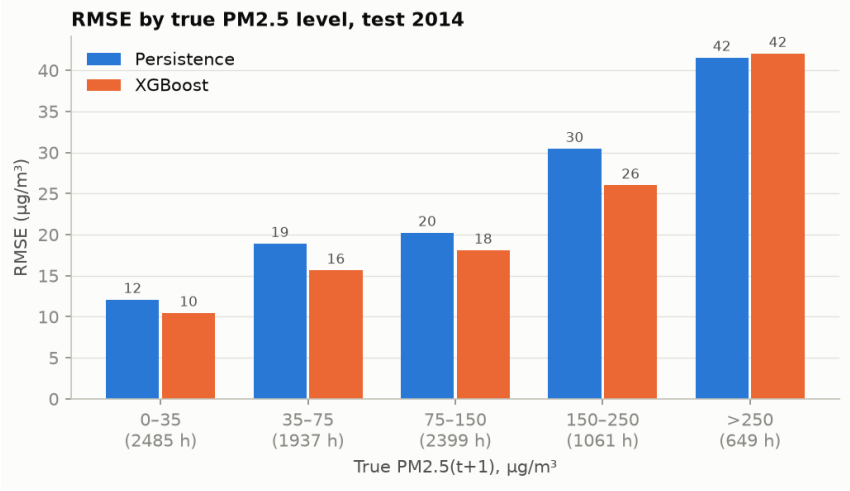


Fig. 7. Test-year RMSE grouped by the true PM2.5 level. XGBoost wins at low-to-moderate levels but not above 250 µg m^{-3}.

inputs use only observed weather; we did not test using operational weather forecasts as inputs. The model does not improve on persistence for the most extreme pollution hours. Finally, the Diebold–Mariano test slightly over-rejected (about 8% at a nominal 5% level) in a simulation with correlated errors, so its p-values should be read as strong but approximate, even though the observed values are far below any usual threshold.

Future scope. Several directions could extend this work: using several stations and more test years for a stronger evaluation; adding weather-forecast inputs so the model can anticipate build-ups; and trying loss functions that pay more attention to extreme values. As planned for Phase 2, sequence models such as LSTMs [14] can also be evaluated under the same protocol to see whether they handle sharp rises and extremes better.

VIII. Conclusion

We built a complete, leakage-controlled pipeline to forecast Beijing PM2.5 one hour ahead and compared an XGBoost model against strong naive baselines. Using 37 features from past observations and a change-based prediction target, XGBoost reduced the test-year error by about 8–9% over persistence, improved in all twelve months, and passed a statistical significance test, while the seasonal-naive baseline proved inadequate. The model's advantage is concentrated at low-to-moderate pollution and disappears during extreme haze, which we explain through its error analysis. Beyond the numbers, the project demonstrates sound machine learning practice for time-series data: honest baselines, strictly chronological evaluation, careful leakage control, and interpretation of where the model works and where it does not. All results are reproducible — re-running the notebooks in a fresh environment regenerated every metric and prediction file exactly.

References

[1] World Health Organization, "WHO global air quality guidelines: particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide," World Health Organization, Geneva, Tech. Rep., 2021, iSBN 9789240034228. [Online]. Available: https://www.who.int/publications/i/item/9789240034228

[2] L. Bai, J. Wang, X. Ma, and H. Lu, "Air pollution forecasts: An overview," International Journal of Environmental Research and Public Health, vol. 15, no. 4, p. 780, 2018.

[3] X. Liang, T. Zou, B. Guo, S. Li, H. Zhang, S. Zhang, H. Huang, and S. X. Chen, "Assessing Beijing's PM2.5 pollution: severity, weather impact, APEC and winter heating," Proceedings of the Royal Society A, vol. 471, no. 2182, p. 20150257, 2015.

[4] S. Chen, "Beijing PM2.5 [dataset]," UCI Machine Learning Repository, 2015, license: CC BY 4.0.

[5] J. Zhong, X. Zhang, K. Gui, Y. Wang, H. Che, X. Shen, L. Zhang, Y. Zhang, J. Sun, and W. Zhang, "Robust prediction of hourly PM2.5 from meteorological data using LightGBM," National Science Review, vol. 8, no. 10, p. nwaa307, 2021.

[6] X. Li, L. Peng, X. Yao, S. Cui, Y. Hu, C. You, and T. Chi, "Long short-term memory neural network for air pollutant concentration predictions: Method development and evaluation," Environmental Pollution, vol. 231, pp. 997–1004, 2017.

[7] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.
[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794.
[9] L. J. Tashman, "Out-of-sample tests of forecasting accuracy: an analysis and review," International Journal of Forecasting, vol. 16, no. 4, pp. 437–450, 2000.
[10] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," Information Sciences, 2012.
[11] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/
[12] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," Journal of Business & Economic Statistics, vol. 13, no. 3, pp. 253–263, 1995.
[13] W. K. Newey and K. D. West, "A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix," Econometrica, vol. 55, no. 3, pp. 703–708, 1987.
[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.